



Remove OuterMost Parenthesis

<https://leetcode.com/problems/remove-outermost-parentheses/>

A valid parentheses string is either empty `""`, `"(" + A + ")"`, or `A + B`, where `A` and `B` are valid parentheses strings, and `+` represents string concatenation.

- For example, `""`, `"()"`, `"(()())"`, and `"(()(()))"` are all valid parentheses strings.

A valid parentheses string `s` is primitive if it is nonempty, and there does not exist a way to split it into `s = A + B`, with `A` and `B` nonempty valid parentheses strings.

Given a valid parentheses string `s`, consider its primitive decomposition:

`s = P1 + P2 + ... + Pk`, where `Pi` are primitive valid parentheses strings.

Return `s` after removing the outermost parentheses of every primitive string in the primitive decomposition of `s`.

Example 1:

Input: `s = "(()())(())"`

Output: `"()()()`

Explanation:

The input string is `"(()())(())"`, with primitive decomposition `"(()())" + "(())"`.

After removing outer parentheses of each part, this is `"()()" + "()" = "()()()"`.

Example 2:

Input: `s = "(()())(())(()())"`

Output: `"()()()()()"`

Explanation:

The input string is `"(()())(())(()())"`, with primitive decomposition `"(()())" + "(())" + "(()())"`.

After removing outer parentheses of each part, this is `"()()" + "()" + "(()())" = "()()()()()"`.

Example 3:

Input: `s = "()()"`

Output: `""`

Explanation:

The input string is "()", with primitive decomposition "(" + ")".
After removing outer parentheses of each part, this is "" + "" = "".

```
class Solution {
public:
    string removeOuterParentheses(string S) {
        string res;
        int opened = 0;
        for (char c : S) {
            if (c == '(' && opened++ > 0) res += c;
            if (c == ')' && opened-- > 1) res += c;
        }
        return res;
    }
};
```

Reverse Words in a Given String

<https://leetcode.com/problems/reverse-words-in-a-string/>

Given an input string `s`, reverse the order of the **words**.

A **word** is defined as a sequence of non-space characters. The **words** in `s` will be separated by at least one space.

Return *a string of the words in reverse order concatenated by a single space*.

Note that `s` may contain leading or trailing spaces or multiple spaces between two words. The returned string should only have a single space separating the words. Do not include any extra spaces.

Example 1:

Input: `s = "the sky is blue"`

Output: `"blue is sky the"`

Example 2:

Input: `s = " hello world "`

Output: "world hello"

Explanation: Your reversed string should not contain leading or trailing spaces.

Example 3:

Input: s = "a good example"

Output: "example good a"

Explanation: You need to reduce multiple spaces between two words to a single space in the reversed string.

```
string reverseWords(string s) {
    if(s.size() == 0) return s;
    stack<string> stack;
    string result;
    for(int i=0; i<s.size(); i++) {
        string word;
        if(s[i]==' ') continue; //skip spaces
        while(i<s.size() && s[i]!=' ') { //store continuous letters into word
            word += s[i]; i++;
        }
        stack.push(word); //push word to the stack
    }
    while(!stack.empty()) {
        result += stack.top(); stack.pop();
        if(!stack.empty()) result += " ";
    }
    return result;
}
```

Largest Odd Number in a String

<https://leetcode.com/problems/largest-odd-number-in-string/>

You are given a string `num` representing a large integer. Return the largest-valued odd integer (as a string) that is a **non-empty** substring of `num`, or an empty string if no odd integer exists.

A substring is a contiguous sequence of characters within a string.

Example 1

Input:

```
num = "52"
```

Output:

```
"5"
```

Explanation:

The only non-empty substrings are "5", "2", and "52". "5" is the only odd number.

Example 2

Input:

```
num = "4206"
```

Output:

```
""
```

Explanation:

There are no odd numbers in "4206".

Example 3

Input:

```
num = "35427"
```

Output:

```
"35427"
```

Explanation:

"35427" is already an odd number.

Constraints

- $1 \leq \text{num.length} \leq 10^5$
- `num` consists only of digits and does not contain any leading zeros.

```
class Solution {
public:
    string largestOddNumber(string num) {
        if (num.back() % 2 == 1) return num;
        int i = num.length() - 1;
        while (i >= 0) {
            int n = num[i];
            if (n % 2 == 1) return num.substr(0, i + 1);
            i--;
        }
        return "";
    }
};
```

Longest Common Prefix

Write a function to find the longest common prefix string amongst an array of strings.

If there is no common prefix, return an empty string `""`.

Example 1:

Input: strs = ["flower", "flow", "flight"]

Output: "fl"

Example 2:

Input: strs = ["dog", "racecar", "car"]

Output: ""

Explanation: There is no common prefix among the input strings.

```
class Solution {
public:
    string longestCommonPrefix(vector<string>& v) {
        string ans="";
        sort(v.begin(),v.end());
        int n=v.size();
        string first=v[0],last=v[n-1];
        for(int i=0;i<min(first.size(),last.size());i++){
            if(first[i]!=last[i]){
                return ans;
            }
            ans+=first[i];
        }
        return ans;
    }
};
```

Isomorphic String

<https://leetcode.com/problems/isomorphic-strings/>

Given two strings `s` and `t`, determine if they are isomorphic.

Two strings `s` and `t` are isomorphic if the characters in `s` can be replaced to get `t`.

All occurrences of a character must be replaced with another character while preserving the

order of characters. No two characters may map to the same character, but a character may map to itself.

Example 1:

Input: s = "egg", t = "add"

Output: true

Example 2:

Input: s = "foo", t = "bar"

Output: false

Example 3:

Input: s = "paper", t = "title"

Output: true

```

class Solution {
public:
    bool isIsomorphic(string s, string t) {
        vector<int> indexS(200, 0); // Stores index of characters in string s
        vector<int> indexT(200, 0); // Stores index of characters in string t

        int len = s.length(); // Get the length of both strings

        if(len != t.length()) { // If the lengths of the two strings are different, they can't be
            return false;
        }

        for(int i = 0; i < len; i++) { // Iterate through each character of the strings
            if(indexS[s[i]] != indexT[t[i]]) { // Check if the index of the current character in s
                return false; // If different, strings are not isomorphic
            }

            indexS[s[i]] = i + 1; // updating position of current character
            indexT[t[i]] = i + 1;
        }

        return true; // If the loop completes without returning false, strings are isomorphic
    }
};

```

Check whether one string is a rotation of another

<https://leetcode.com/problems/rotate-string/>

Given two strings `s` and `goal`, return `true` if and only if `s` can become `goal` after some number of **shifts** on `s`.

A **shift** on `s` consists of moving the leftmost character of `s` to the rightmost position.

- For example, if `s = "abcde"`, then it will be `"bcdea"` after one shift.

Example 1:

Input: `s = "abcde"`, `goal = "cdeab"`

Output: true

Example 2:

Input: s = "abcde", goal = "abced"

Output: false

```
bool rotateString(string A, string B) {  
    return A.size() == B.size() && (A + A).find(B) != string::npos;  
}
```

Check whether 2 strings are anagrams of one another

<https://leetcode.com/problems/valid-anagram>

Given two strings `s` and `t`, return `true` if `t` is an anagram of `s`, and `false` otherwise.

An **Anagram** is a word or phrase formed by rearranging the letters of a different word or phrase, typically using all the original letters exactly once.

Example 1:

Input: s = "anagram", t = "nagaram"

Output: true

Example 2:

Input: s = "rat", t = "car"

Output: false

Constraints:

- `1 <= s.length, t.length <= 5 * 104`
- `s` and `t` consist of lowercase English letters.

```

class Solution {
public:
    bool isAnagram(string s, string t) {
        if(s.length() != t.length()){
            return false;
        }
        int arr[26] = {0};
        for(int i=0; i<s.length(); i++){
            arr[s[i]-'a']++;
            arr[t[i]-'a']--;
        }
        for(int i=0; i<26; i++){
            if(arr[i] != 0){
                return false;
            }
        }
        return true;
    }
};

```

Sort characters by frequency

<https://leetcode.com/problems/sort-characters-by-frequency>

Given a string `s`, sort it in **decreasing order** based on the **frequency** of the characters. The **frequency** of a character is the number of times it appears in the string.

Return *the sorted string*. If there are multiple answers, return *any of them*.

Example 1:

Input: `s = "tree"`

Output: `"eert"`

Explanation: 'e' appears twice while 'r' and 't' both appear once.

So 'e' must appear before both 'r' and 't'. Therefore "eetr" is also a valid answer.

Example 2:

Input: `s = "cccaaa"`

Output: "aaaccc"

Explanation: Both 'c' and 'a' appear three times, so both "cccaaa" and "aaaccc" are valid answers.

Note that "cacaca" is incorrect, as the same characters must be together.

Example 3:

Input: s = "Aabb"

Output: "bbAa"

Explanation: "bbaA" is also a valid answer, but "Aabb" is incorrect.

Note that 'A' and 'a' are treated as two different characters.

```
class Solution {
public:
    string frequencySort(string s) {
        unordered_map<char,int> mp;
        multimap<int,char> r;
        string ss="";

        for(auto a : s)
            mp[a]++;

        for(auto a : mp)
            r.insert({a.second, a.first});

        for(auto it = r.rbegin(); it != r.rend(); ++it)
            ss += string(it->first, it->second);

        return ss;
    }
};
```

Maximum Nesting Depth of Parenthesis

<https://leetcode.com/problems/maximum-nesting-depth-of-the-parentheses/>

Given a **valid parentheses string** `s`, return the **nesting depth** of `s`. The nesting depth is the **maximum** number of nested parentheses.

Example 1:

Input: $s = "(1+(2*3)+((8)/4))+1"$

Output: 3

Explanation:

Digit 8 is inside of 3 nested parentheses in the string.

Example 2:

Input: $s = "(1)+((2))+(((3)))"$

Output: 3

Explanation:

Digit 3 is inside of 3 nested parentheses in the string.

Example 3:

Input: $s = "()()((()()))"$

Output: 3

```

class Solution {
public:
    int maxDepth(string s) {
        int count = 0;
        int max_num = 0;
        for (char c : s) {
            if (c == '(') {
                count++;
                if (max_num < count)
                    max_num = count;
            } else if (c == ')') {
                count--;
            }
        }
        return max_num;
    }
};

```

Roman Number to Integer

<https://leetcode.com/problems/roman-to-integer>

Roman numerals are represented by seven different symbols: **I**, **V**, **X**, **L**, **C**, **D** and **M**.

Symbol Value

I 1

V 5

X 10

L 50

C 100

D 500

M 1000

For example, **2** is written as **II** in Roman numeral, just two ones added together. **12** is written as **XII**, which is simply **X** + **II**. The number **27** is written as **XXVII**, which is **XX** + **V** + **II**.

Roman numerals are usually written largest to smallest from left to right. However, the numeral

for four is not `IIII` . Instead, the number four is written as `IV` . Because the one is before the five we subtract it making four. The same principle applies to the number nine, which is written as `IX` . There are six instances where subtraction is used:

- `I` can be placed before `V` (5) and `X` (10) to make 4 and 9.
- `X` can be placed before `L` (50) and `C` (100) to make 40 and 90.
- `C` can be placed before `D` (500) and `M` (1000) to make 400 and 900.

Given a roman numeral, convert it to an integer.

Example 1:

Input: `s = "III"`

Output: 3

Explanation: `III` = 3.

Example 2:

Input: `s = "LVIII"`

Output: 58

Explanation: `L` = 50, `V` = 5, `III` = 3.

Example 3:

Input: `s = "MCMXCIV"`

Output: 1994

Explanation: `M` = 1000, `CM` = 900, `XC` = 90 and `IV` = 4.

Constraints:

- `1 <= s.length <= 15`
- `s` contains only the characters `('I', 'V', 'X', 'L', 'C', 'D', 'M')` .
- It is **guaranteed** that `s` is a valid roman numeral in the range `[1, 3999]` .

```
/*
```

Time Complexity : $O(1)$, The maximum length of the string(s) can be 15 (as per the Constraint) worst case time complexity can be $O(15)$ or $O(1)$.

Space Complexity : $O(1)$, We are using unordered_map(map) to store the Roman symbols and their integer values but there are only 7 symbols hence the worst case space complexity can be $O(7)$ equivalent to $O(1)$.

Solved using String + Hash Table + Math.

```
*/
```

```
class Solution {
public:
    int romanToInt(string s) {
        unordered_map<char, int> map;

        map['I'] = 1;
        map['V'] = 5;
        map['X'] = 10;
        map['L'] = 50;
        map['C'] = 100;
        map['D'] = 500;
        map['M'] = 1000;

        int ans = 0;

        for(int i=0; i<s.length(); i++){
            if(map[s[i]] < map[s[i+1]]){
                ans -= map[s[i]];
            }
            else{
                ans += map[s[i]];
            }
        }
        return ans;
    }
};
```

Implement ATOI

<https://leetcode.com/problems/string-to-integer-atoi/>

Implement the `myAtoi(string s)` function, which converts a string to a 32-bit signed integer.

The algorithm for `myAtoi(string s)` is as follows:

1. **Whitespace:** Ignore any leading whitespace (" ").
2. **Signedness:** Determine the sign by checking if the next character is '-' or '+', assuming positivity is neither present.
3. **Conversion:** Read the integer by skipping leading zeros until a non-digit character is encountered or the end of the string is reached. If no digits were read, then the result is 0.
4. **Rounding:** If the integer is out of the 32-bit signed integer range $[-2^{31}, 2^{31} - 1]$, then round the integer to remain in the range. Specifically, integers less than -2^{31} should be rounded to -2^{31} , and integers greater than $2^{31} - 1$ should be rounded to $2^{31} - 1$.

Return the integer as the final result.

Example 1:

Input: `s = "42"`

Output: 42

Explanation:

The underlined characters are what is read in and the caret is the current reader position.

Step 1: "42" (no characters read because there is no leading whitespace)

^

Step 2: "42" (no characters read because there is neither a '-' nor '+')

^

Step 3: "42" ("42" is read in)

^

Example 2:

Input: `s = "-042"`

Output: -42

Explanation:

Step 1: " -042" (leading whitespace is read and ignored)

^

Step 2: " -042" ('-' is read, so the result should be negative)

^

Step 3: " -042" ("042" is read in, leading zeros ignored in the result)

^

Example 3:

Input: s = "1337c0d3"

Output: 1337

Explanation:

Step 1: "1337c0d3" (no characters read because there is no leading whitespace)

^

Step 2: "1337c0d3" (no characters read because there is neither a '-' nor '+')

^

Step 3: "1337c0d3" ("1337" is read in; reading stops because the next character is a non-digit)

^

Example 4:

Input: s = "0-1"

Output: 0

Explanation:

Step 1: "0-1" (no characters read because there is no leading whitespace)

^

Step 2: "0-1" (no characters read because there is neither a '-' nor '+')

^

Step 3: "0-1" ("0" is read in; reading stops because the next character is a non-digit)

^

Example 5:

Input: s = "words and 987"

Output: 0

Explanation:

Reading stops at the first non-digit character 'w'.

```
/*
```

Time Complexity : $O(\log N)$, Since we are going through the entire number digit by digit, the time complexity should be $O(\log_{10} N)$. The reason behind $\log_{10}$ is because we are dealing with integers which are in base 10.

Space Complexity : $O(1)$, We are not using any data structure for interim operations, therefore the space complexity is $O(1)$.

Solved using String.

```
*/
```

```
class Solution {
public:
    int myAtoi(string s) {
        int len = s.size();
        double num = 0;
        int i=0;
        while(s[i] == ' '){
            i++;
        }
        bool positive = s[i] == '+';
        bool negative = s[i] == '-';
        positive == true ? i++ : i;
        negative == true ? i++ : i;
        while(i < len && s[i] >= '0' && s[i] <= '9'){
            num = num*10 + (s[i]-'0');
            i++;
        }
        num = negative ? -num : num;
        cout<<num<<endl;
        num = (num > INT_MAX) ? INT_MAX : num;
        num = (num < INT_MIN) ? INT_MIN : num;
        cout<<num<<endl;
        return int(num);
    }
};
```

Count Number of substrings

<https://bit.ly/3CfQfYi>

Given a string of lowercase alphabets, count all possible **substrings** (not necessarily distinct) that have **exactly k distinct** characters.

Example 1:

Input: S = "aba", K = 2

****Output:****3

Explanation: The substrings are: "ab", "ba" and "aba".

Input: S = "abaaca", K = 1

Output: 7

Explanation: The substrings are: "a", "b", "a", "aa", "a", "c", "a".

```

//User function template for C++

class Solution
{
public:
    long long int solve(string s , int k)
    {
        long long ans = 0;
        int cnt = 0;
        int i = 0 , j = 0;
        int size = s.size();
        vector<int>mp(26 , 0);

        while(j < size)
        {
            mp[s[j] - 'a']++;
            if(mp[s[j] - 'a'] == 1) cnt++;
            while(cnt > k)
            {
                mp[s[i] - 'a']--;
                if(mp[s[i] - 'a'] <= 0)
                {
                    cnt--;
                }

                i++;
            }
            ans += j-i+1;
            j++;
        }
        return ans;
    }
    long long int substrCount (string s, int k)
    {
        return solve(s , k) - solve(s , k-1);
    }
};

```

Longest Palindromic Substring (no DP)

<https://leetcode.com/problems/longest-palindromic-substring/>

Given a string `s`, return *the longest*

palindromic

substring

in `s`.

Example 1:

Input: `s = "babad"`

Output: `"bab"`

Explanation: `"aba"` is also a valid answer.

Example 2:

Input: `s = "cbabd"`

Output: `"bb"`

The non DP version uses the expand-around-center approach:

```

class Solution {
public:
    std::string longestPalindrome(std::string s) {
        if (s.empty()) return "";
        int start = 0, end = 0;
        for (int i = 0; i < s.size(); ++i) {
            // Odd length palindromes
            int left = i, right = i;
            while (left >= 0 && right < s.size() && s[left] == s[right]) {
                if (right - left > end - start) {
                    start = left;
                    end = right;
                }
                --left;
                ++right;
            }
            // Even length palindromes
            left = i, right = i + 1;
            while (left >= 0 && right < s.size() && s[left] == s[right]) {
                if (right - left > end - start) {
                    start = left;
                    end = right;
                }
                --left;
                ++right;
            }
        }
        return s.substr(start, end - start + 1);
    }
};

```

Explanation

- **Expand Around Center:**

For each index `i`, we treat it as the center of a potential palindrome and expand outwards for both odd and even length palindromes.

- **Update Longest Palindrome:**

If a longer palindrome is found during expansion, we update `start` and `end` accordingly.

- **Result:**

Finally, return the substring defined by the updated indices.

Manacher's Algorithm

```

class Solution {
public:
    std::string longestPalindrome(std::string s) {
        if (s.length() <= 1) {
            return s;
        }

        int maxLen = 1;
        std::string maxStr = s.substr(0, 1);
        s = "#" + std::regex_replace(s, std::regex(""), "#") + "#";
        std::vector<int> dp(s.length(), 0);
        int center = 0;
        int right = 0;

        for (int i = 0; i < s.length(); ++i) {
            if (i < right) {
                dp[i] = std::min(right - i, dp[2 * center - i]);
            }

            while (i - dp[i] - 1 >= 0 && i + dp[i] + 1 < s.length() && s[i - dp[i] - 1] == s[i + dp[i] + 1]) {
                dp[i]++;
            }

            if (i + dp[i] > right) {
                center = i;
                right = i + dp[i];
            }

            if (dp[i] > maxLen) {
                maxLen = dp[i];
                maxStr = s.substr(i - dp[i], 2 * dp[i] + 1);
                maxStr.erase(std::remove(maxStr.begin(), maxStr.end(), '#'), maxStr.end());
            }
        }

        return maxStr;
    }
};

```

Sum of beauty of all substrings

<https://leetcode.com/problems/sum-of-beauty-of-all-substrings/>

The **beauty** of a string is the difference in frequencies between the most frequent and least frequent characters.

- For example, the beauty of "abaacc" is $3 - 1 = 2$.

Given a string `s`, return *the sum of **beauty** of all of its substrings*.

Example 1:

Input: `s = "aabcb"`

Output: 5

Explanation: The substrings with non-zero beauty are ["aab", "aabc", "aabcb", "abcb", "bcb"], each with beauty equal to 1.

Example 2:

Input: `s = "aabcbaa"`

Output: 17

```

class Solution {
public:
    int beauty(const std::vector<int> &mapy) {
        int mf = INT_MIN; // max frequency
        int lf = INT_MAX; // min frequency (excluding zeros)

        for(int i = 0; i < 26; i++) {
            if(mapy[i] > 0) {
                if(mapy[i] > mf) {
                    mf = mapy[i];
                }
                if(mapy[i] < lf) {
                    lf = mapy[i];
                }
            }
        }

        return (mf - lf);
    }

    int beautySum(const std::string &s) {
        int res = 0;

        for(int i = 0; i < s.length(); i++) {
            std::vector<int> mapy(26, 0);
            for(int j = i; j < s.length(); j++) {
                mapy[s[j] - 'a']++;
                res += beauty(mapy);
            }
        }

        return res;
    }
};

```

but this is bruteforce, we can use prefix sums and DP:

Below is a dynamic programming version that precomputes prefix frequency counts so that you can compute the frequency of any substring in $O(26)$ time:

```

class Solution {
public:
    int beautySum(string s) {
        int n = s.size();
        int result = 0;

        // Build a prefix frequency array:
        // prefix[i+1] stores the frequency counts for substring s[0...i]
        vector<vector<int>> prefix(n + 1, vector<int>(26, 0));
        for (int i = 0; i < n; ++i) {
            prefix[i + 1] = prefix[i];           // Copy previous counts
            prefix[i + 1][s[i] - 'a']++;         // Update count for current character
        }

        // Iterate over all substrings s[i...j]
        for (int i = 0; i < n; ++i) {
            for (int j = i; j < n; ++j) {
                int maxFreq = 0, minFreq = INT_MAX;
                // Compute frequency for each character using the prefix array:
                // The frequency of character c in s[i...j] is:
                //   prefix[j+1][c] - prefix[i][c]
                for (int c = 0; c < 26; ++c) {
                    int count = prefix[j + 1][c] - prefix[i][c];
                    if (count > 0) {
                        maxFreq = max(maxFreq, count);
                        minFreq = min(minFreq, count);
                    }
                }
                result += (maxFreq - minFreq);
            }
        }
        return result;
    }
};

```

Explanation

1. Prefix Frequency Array Construction:

- A 2D vector `prefix` is built such that `prefix[i+1]` holds the frequency counts for all

characters in the substring `s[0...i]`.

- This is built iteratively by copying the previous frequency counts and updating the count for the current character.

2. Computing Frequencies for a Substring:

- For any substring `s[i...j]`, the frequency of a character `c` is obtained by subtracting:

```
count = prefix[j+1][c] - prefix[i][c]
```

- This avoids recomputing the frequency counts from scratch for every substring.

3. Calculating the Beauty:

- For each substring, the maximum (`maxFreq`) and minimum (`minFreq`) frequencies (ignoring zeros) are computed.
- The beauty of the substring is `maxFreq - minFreq`, which is added to the final result.

This DP approach leverages precomputed information to optimize repeated frequency calculations, resulting in a cleaner and more efficient solution compared to recomputing frequencies for each substring directly.